

Relazione: Lavorare con File di Testo nella CLI

Sei riuscito a trovare subito space.txt?

No. Anche se SciTE si trova nella directory corretta (`/home/analyst`), `space.txt` non viene visualizzato inizialmente perché l'editor cerca estensioni note e `.txt` non è tra queste; per vederlo è necessario selezionare l'opzione All Files (*) dal menu a discesa.

Quale carattere usa nano per rappresentare che una linea continua oltre i bordi dello schermo?

Un simbolo di maggiore (`>`) evidenziato.

Perché i file di configurazione delle applicazioni utente sono salvati nella directory home dell'utente e non sotto `/etc` con tutti gli altri file di configurazione a livello di sistema?

Perché la directory home permette all'utente (non amministratore/root) di modificare le proprie preferenze e personalizzazioni in autonomia, mentre la cartella `/etc` contiene i file di configurazione globali del sistema, protetti e accessibili in scrittura solo dall'utente `root`.

Anche la finestra del terminale che era già aperta ha cambiato colore da verde a rosso? Spiega.

No, la finestra del terminale già aperta non cambia colore automaticamente. Il file `.bashrc` viene letto ed eseguito dalla shell esclusivamente nel momento in cui viene avviata una nuova sessione o un nuovo terminale, mentre le finestre già aperte mantengono in memoria le impostazioni caricate in precedenza. Per applicare immediatamente le modifiche senza chiudere la finestra corrente, è necessario ricaricare manualmente il file eseguendo il comando `source ~/.bashrc`.

A cosa si riferisce il messaggio di errore?

Il messaggio di errore si riferisce a un problema di **sintassi** nel file di configurazione di nginx. Nel blocco di configurazione, la rimozione accidentale del punto e virgola (`;`) alla fine di una riga di istruzione impedisce a nginx di interpretare correttamente i parametri. All'avvio del servizio, il parser del server web rileva l'incoerenza formale e genera un errore, bloccando l'esecuzione.

Appare la pagina web?

No, la pagina web non appare. Dopo aver arrestato il server nginx con il comando `sudo kill nginx`, il servizio non è più attivo sulla porta 8080. Di conseguenza, tentando di raggiungere l'indirizzo `127.0.0.1:8080` nel browser, Firefox mostrerà un errore di connessione (come "Impossibile connettersi" o "Connessione rifiutata"), confermando che il server web è stato spento correttamente.

Puoi modificare il file `/etc/nginx/custom_server.conf` con SciTE? Descrivi il processo di seguito.

Sì, è possibile modificare il file con SciTE avviando l'applicazione dal terminale con i privilegi di amministratore tramite il comando `sudo scite`. Una volta aperta l'interfaccia grafica, si seleziona **File > Open**, si naviga fino al percorso `/etc/nginx/` (impostando eventualmente il filtro su *All Files*), si individua e si apre `custom_server.conf`. Dopo aver apportato le modifiche desiderate mantenendo la corretta sintassi, si procede al salvataggio tramite **File > Save** e infine si chiude il programma, certi che i permessi di root eviteranno errori di accesso negato.

BONUS: Prendere Familiarità con la Shell Linux

Elenca alcune sezioni incluse in una pagina man.

Le sezioni tipicamente incluse in una pagina di manuale (`man`) in Linux comprendono **NAME**, **OPTIONS**, **BUGS** e **HISTORY**.

Qual è la funzione del comando `cp`?

Il comando `cp` viene utilizzato per copiare file e directory da una posizione a un'altra all'interno del file system.

Quale comando useresti per trovare maggiori informazioni sul comando `pwd`? Qual è la funzione del comando `pwd`?

Per trovare maggiori informazioni sul comando `pwd`, useresti il comando `man pwd`. Il comando `pwd` (Print Working Directory) serve a visualizzare il percorso assoluto della directory corrente in cui ci si trova in quel momento nella shell.

Qual è la directory corrente?

`/home/analyst`

In quale cartella ti trovi ora?

`/home/analyst/cyops_folder3`

Perché è successo?

Cosa succede: La shell sposta immediatamente la directory corrente alla directory home dell'utente (nel nostro caso `/home/analyst`) e il prompt dei comandi si aggiorna mostrando la tilde (es. `[analyst@sec0ps ~]$`).

Perché è successo: Il comando `cd` (Change Directory) viene utilizzato per navigare tra le cartelle, mentre il carattere tilde (`~`) è una scorciatoia standard dei sistemi Linux che

rappresenta la directory home dell'utente attualmente autenticato, indipendentemente da quale cartella ci si trovi in quel momento.

Digita `cd .` e cosa succede?

Non si verifica alcun cambiamento visibile nella directory corrente. La shell rimane esattamente nella cartella in cui ci si trovava prima (`/home/analyst/cyops_folder3`), poiché il punto singolo (`.`) punta alla directory corrente stessa.

Cosa succede eseguendo `cd ..` da `/home/analyst/cyops_folder3`?

La shell sposta la directory corrente al livello superiore (la directory genitore), portandoti in `/home/analyst`.

Quale sarebbe la directory corrente se eseguiessi il comando `cd ..` da `/home/analyst` (rappresentata da `~`)?

La directory corrente diventerebbe `/home`.

Quale sarebbe la directory corrente se eseguiessi il comando `cd ..` da `/home`?

La directory corrente diventerebbe la directory radice `/`.

Quale sarebbe la directory corrente se eseguiessi il comando `cd ..` dalla directory radice `/`?

La directory corrente rimarrebbe sempre la directory radice `/`, poiché non esistono livelli superiori nel file system Linux.

Nessun output è stato mostrato. È previsto? Spiega.

Sì, è perfettamente previsto. L'operatore di redirectione (`>`) modifica la destinazione dell'output generato dal comando `echo`: anziché inviare il testo allo schermo del terminale (comportamento predefinito), lo cattura e lo scrive all'interno del file specificato (`some_text_file.txt`). Di conseguenza, la console rimane vuota durante l'esecuzione dell'operazione.

Cosa è successo al file di testo? Spiega.

Il contenuto precedente del file `some_text_file.txt` è stato completamente sovrascritto dal nuovo messaggio. L'operatore di redirectione semplice (`>`) ha come comportamento predefinito quello di azzerare il file esistente (o crearne uno nuovo se non esiste) prima di scrivervi il nuovo output, eliminando irrimediabilmente i dati memorizzati in precedenza.

Cosa è successo al file di testo? Spiega.

Il contenuto precedente del file è rimasto completamente intatto e la nuova frase è stata aggiunta (accodata) alla fine del documento. L'operatore di redirectione doppio (`>>`) evita la

sovrascrittura tipica del singolo `>`, preservando i dati esistenti e inserendo il nuovo output in coda.

Quanti file vengono visualizzati eseguendo `ls -l` nella directory home senza opzioni aggiuntive?

Vengono visualizzati solo i file e le cartelle standard (visibili), escludendo completamente i file e le directory che iniziano con un punto (i file nascosti), poiché l'opzione `-l` da sola non mostra gli elementi nascosti a meno che non si utilizzi il flag `-a` (come in `ls -la`).

Quanti file in più vengono visualizzati rispetto a prima? Spiega.

Vengono visualizzati tutti i file e le directory che iniziano con un punto (i file nascosti), insieme alle directory speciali `.` e `..`. L'aumento esatto dipende dai file di configurazione presenti nella home dell'utente, ma il motivo di questa differenza è che il comando `ls -l` standard ignora i file nascosti per impostazione predefinita, mentre l'aggiunta dell'opzione `-a` (come in `ls -la`) forza la visualizzazione dell'intero contenuto senza alcuna esclusione.

È possibile nascondere intere directory aggiungendo un punto prima del loro nome? Ci sono directory nascoste nell'output di `ls -la` sopra?

Sì, è assolutamente possibile nascondere intere directory anteponendo un punto al loro nome. In Linux, qualsiasi cartella il cui nome inizia con un punto (ad esempio `.cartella_nascosta`) viene considerata invisibile ai comandi di visualizzazione standard come `ls`, a meno che non venga utilizzato esplicitamente il flag `-a` per rivelarne la presenza.

Sì, ci sono directory nascoste nell'output di `ls -la`.

Nello specifico, l'output include:

- **Le directory speciali `.` e `..`:** Che rappresentano rispettivamente la directory corrente e la directory genitore.
- **Le cartelle con il punto iniziale:** Qualsiasi directory di sistema o personalizzata il cui nome inizia con un punto (come `.config`, `.cache` o eventuali cartelle di laboratorio nascoste), identificabili nella prima colonna dei permessi dalla lettera `d`.

Fornisci tre esempi di file nascosti mostrati nell'output di `ls -la` sopra.

`.file5`

`.file6`

`.file7`

Quali sono i file sorgente e destinazione? (usa percorsi completi per rappresentare i parametri)

File sorgente: `/home/analyst/some_text_file.txt`

File/Directory destinazione: `/home/analyst/cyops_folder2/`

Quale comando hai usato per completare l'attività?

```
mv /home/analyst/cyops_folder2/some_text_file.txt ~
```

Quali sono i vantaggi dell'utilizzo della riga di comando Linux?

Efficienza e velocità: Permette di eseguire operazioni complesse, ricerche o modifiche massive su file e cartelle in pochi secondi tramite scorciatoie e comandi mirati, risultando molto più rapida rispetto all'interfaccia grafica.

Automazione e scripting: Consente di combinare i comandi in script Bash per automatizzare attività ripetitive, backup o report di sicurezza, riducendo al minimo l'intervento manuale.

Controllo granulare: Offre accesso diretto e profondo alle configurazioni di sistema, ai permessi dei file e ai processi in esecuzione, garantendo una precisione che le interfacce visive spesso non consentono.

Leggerezza e consumo di risorse: Non richiede l'uso di un ambiente grafico (GUI), consumando pochissima memoria RAM e potenza di calcolo, aspetto ideale per gestire server remoti o macchine virtuali.

Standardizzazione e portabilità: Le competenze acquisite sulla riga di comando sono universali e funzionano su quasi tutte le distribuzioni Linux, su server headless e su sistemi Unix-like.